Planning as Autoregressive Sequence Modelling: Decoding Strategies

Dian Kruger
Shane Josias

Introduction

Planning is a core problem in artificial intelligence: given a model of an environment and a desired objective, an agent must construct a sequence of actions that leads from an initial state to a goal state while satisfying constraints and optimising some notion of cost or reward. Classical approaches formulate planning as search over a state space (e.g., A*) [1], whereas reinforcement learning (RL) frames it as maximising cumulative reward through value functions or policies [2]. More recently, a generative modelling perspective has emerged, in which trajectories themselves are treated as structured sequences that can be modelled probabilistically. From this viewpoint, planning becomes a problem of inference in a learned distribution over trajectories rather than explicit search in a hand-designed state graph [3].

In this project, planning in a 2D gridworld is formulated as autoregressive sequence modelling. A trajectory is represented as an ordered sequence of states (or state–action pairs), and a model is trained using next-step prediction on a dataset of valid trajectories. [1]. Once trained, the model defines a learned probability distribution over feasible trajectories. Planning then corresponds to decoding: generating a trajectory from this distribution that satisfies the task objective. In this framing it is useful to distinguish between autoregressive generation and decoding. Autoregressive generation means that the model produces a trajectory sequentially, predicting each next state (or state–action token) conditioned on the previously generated tokens. Decoding refers to the rule used during inference to convert the model's predicted probability distribution into an actual trajectory. Decoding is necessary because multiple continuations are typically possible at each step, and a procedure is required to decide which one forms the final plan [3].

The central question of this project is how different decoding strategies affect the quality, reliability, diversity, and computational cost of the generated plans. While the underlying model is fixed after training, inference procedures such as greedy decoding, stochastic sampling with temperature control, and beam search may extract substantially different trajectories from the same distribution [4]. Greedy decoding prioritises local likelihood, potentially sacrificing global optimality or diversity. Stochastic sampling can increase diversity but may introduce instability or suboptimal paths. Beam search balances exploration and exploitation at the cost of increased computation [4]. By systematically

comparing these strategies, the project investigates how inference choices shape planning outcomes when planning is cast as generative sequence modelling.

Background

Generative Models

Generative models are types of machine learning systems that aim to represent a probability distribution over data, or to draw samples from that distribution [5]. Specifically, if $\mathbf{x}$ is a collection of features observed from an object or event, like words in a sentence or coordinates in a maze, and $p(\mathbf{x})$ is the true joint distribution of those features, a generative model learns an approximation $\hat{p}(\mathbf{x})$ of this distribution from training observations [5].

Once such a distribution has been learned, it can be used to generate new observations that are similar to the observations in the training data by sampling from $\hat{p}(\mathbf{x})$ [5].

In practice, assumptions are often made to simplify the learning problem, such as assuming a particular parametric form for the distribution (e.g., Gaussian), so instead of learning an arbitrary distribution, the model learns a specific family of distributions parameterised by θ. The learning process then involves finding the parameters θ that best fit the training data [5].

We thus assume that the data's true distribution $p(\mathbf{x})$ belongs to a family of distributions $\{p_\theta(\mathbf{x})\}$, and the model learns the parameters θ that maximise the likelihood of the training data under $p_\theta(\mathbf{x})$.

The Planning Problem

Broadly, automated planning is the field of research addressing the problem of finding an ordered or partially ordered series of actions that changes an environment from its initial state to a desired goal state [6].

In this work, the planning problem is to generate a valid trajectory from a given starting position to a specified goal position in a 2D Maze environment.

In this environment, a trajectory refers to a sequence of maze positions connecting a start location to a goal location without passing through walls. Assuming a discrete grid-based representation, maze coordinates are integers identifying grid cells. A trajectory can therefore be represented as a sequence of coordinates in which each consecutive pair corresponds to adjacent non-wall cells, with adjacency restricted to horizontal and vertical moves.

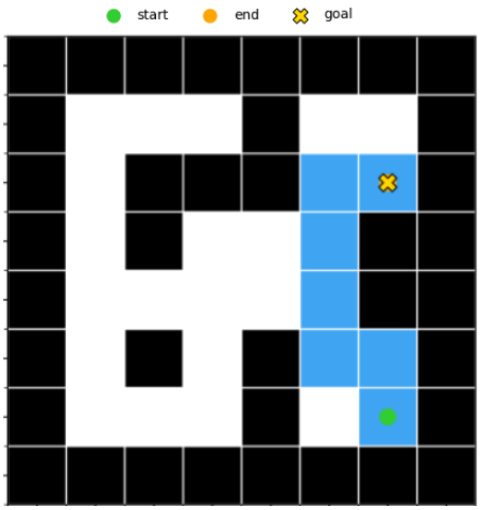

Figure 1: Example of a valid trajectory in a discrete 2D maze environment

Generative Modelling for Planning

If a dataset of valid trajectories is available, a generative model can be trained to learn the distribution over these trajectories. Once trained, the model can be used to generate new trajectories by sampling from the learned distribution. This approach treats planning as a problem of inference in a probabilistic model [3]. So if we define a trajectory $\boldsymbol{\tau}$ with T steps as:

$$\boldsymbol{\tau} = (s_1, a_1, s_2, a_2, \ldots, s_T, a_T)$$

where s_t is the state (coordinates) at time step t and a_t is the action (moving up, down, left or right) taken at time step t, the generative model learns to approximate the distribution $p(\boldsymbol{\tau})$ over valid trajectories. Instead of directly modelling $p(\boldsymbol{\tau})$, an autoregressive model factorises this distribution into a product of conditional probabilities [5]:

$$p(\boldsymbol{\tau}) = \prod_{t=1}^{T} p(z_t \mid z_{<t})$$

where z_t denotes the t^{th} token in the trajectory sequence, which may correspond to either a state or an action, and $z_{<t}$ denotes the sequence of preceding tokens up to z_{t-1}. This simplifies the supervised learning problem to estimating $p(z_t \mid z_{<t})$, so given a prefix of trajectory tokens, predict the probability of the next token in the sequence.

A complete trajectory is then generated autoregressively by repeatedly predicting the next token until a full plan is produced, with the decoding strategy determining how each next-token choice is made. The choice of decoding strategy is important because the model's predicted distribution may assign non-negligible probability mass to multiple valid continuations at each step, and the continuation selected during decoding can significantly affect both the quality of the final trajectory and the computational cost of generation [3]. Since the learned distribution is only an approximation to the true trajectory distribution, it may also assign non-negligible probability to invalid continuations. As a result, the decoding strategy can influence whether a valid trajectory is generated at all. Even when

multiple valid trajectories are possible, some may be more desirable than others, for example because they require fewer steps, and the decoding strategy can therefore affect whether a more optimal trajectory is produced [3, 4].

Decoding Methods

Direct sampling from $p(\boldsymbol{\tau})$ is challenging, since the number of possible trajectories grows exponentially with trajectory length, so explicitly calculating and storing the probabilities for all complete trajectories quickly becomes computationally infeasible [5]. This creates a need for decoding algorithms to approximately sample from, or search within $p(\boldsymbol{\tau})$.

Greedy Decoding

Greedy decoding is the simplest method for generating sequences from $p(\boldsymbol{\tau})$ [4]. At each step t during the autoregressive generation of a trajectory, the algorithm selects the next token $\hat{z}_t$ as follows:

$$\hat{z}_t = \arg\max_{z \in V} p(z \mid z_{<t})$$

where V is the set of all possible tokens, and $z_{<t}$ denotes the values obtained in the previous steps of greedy decoding. Greedy decoding thus always selects the locally optimal choice for the next token in the sequence, meaning it chooses the token with the highest conditional probability given the current prefix [4]. Greedy decoding is therefore deterministic and computationally efficient, but does not necessarily find the globally most probable trajectory, which is the complete trajectory with the highest probability under $p(\boldsymbol{\tau})$ [4].

Beam Search

Beam search extends greedy decoding by retaining multiple candidate sequences at each step rather than only the single most likely continuation [4]. The number of retained candidates is fixed in advance and is known as the beam width B. At each stage, all valid one-step continuations of the current candidates are scored autoregressively under the model, and the top B highest-scoring sequences are kept. This process is repeated until termination, after which the highest-scoring complete trajectory among the retained candidates is returned.

The beam search algorithm in the context of trajectory planning can be described as follows:

The score $\log p_\theta(\boldsymbol{\tau})$ can be accumulated autoregressively for each candidate trajectory. Since the beam width places an upper bound on the number of candidate trajectories in C_t and retained trajectories in $\mathcal{T}_t$ at each step t, maintaining these scores for all candidates remains computationally feasible when the beam width is sufficiently small [4].

Larger beam widths allow the algorithm to consider a greater number of candidate trajectories. As a result, the trajectory returned by beam search will typically be at least as highly scored under the model as the trajectory returned with a smaller beam width, with

Beam Search [3]

Require maximum sequence length T, beam width B, starting position s_{start}, and goal position s_{goal}

Initialize $\mathcal{T}_0 = \{(s_{\text{start}})\}$

for $t = 0, \ldots, T-1$ **do**

  For each partial trajectory $\boldsymbol{\tau} \in \mathcal{T}_t$, let $\mathcal{V}(\boldsymbol{\tau})$ denote the set of valid next tokens available from the final state of $\boldsymbol{\tau}$

  $C_{t+1} \leftarrow \{\, \text{append}(\boldsymbol{\tau}, z) \mid \boldsymbol{\tau} \in \mathcal{T}_t \text{ and } z \in \mathcal{V}(\boldsymbol{\tau}) \,\}$
  // candidate single-token extensions

  $\mathcal{T}_{t+1} \leftarrow$ the set of the B highest-scoring sequences in C_{t+1} under $\log p_\theta(\boldsymbol{\tau})$

  if any $\boldsymbol{\tau} \in \mathcal{T}_{t+1}$ ends at s_{goal} **then stop**

end for

Return $\arg\max_{\boldsymbol{\tau} \in \mathcal{T}_{t+1}} \log p_\theta(\boldsymbol{\tau})$

Figure 2: Pseudocode for beam search in the trajectory planning setting.

a beam width of one corresponding to greedy decoding. This broader search, however, requires the model scores of more candidate trajectories to be computed and maintained, so increasing the beam width also increases the computational cost of decoding [4].

Transformer Architecture

The Transformer architecture is a type of neural network that has become the standard for sequence modelling tasks, notably in natural language processing [4]. Even though the original Transformer was designed for language, the fact that it models sequences of tokens makes it applicable to our trajectory modelling problem, where the tokens could be states and/or actions in a trajectory [3].

The key components of the Transformer architecture are now briefly introduced. For intuition, the discussion refers primarily to natural language processing examples, where the role of these components is often easier to interpret than in trajectory modelling.

1. Input Representation:

Before raw data enters the Transformer, it must be converted into a high dimensional vector representation.

- **Tokenisation**: The first step involves tokenisation, where the input sequence is broken down into discrete tokens. In an NLP scenario, this could invlolde splitting text into words, subwords or characters [4]. For the purposes of this discussion, it is assumed that the input has been tokenised at the word level.
- **Embeddings** (E): Each token ID is used to index into an embedding matrix E, which maps the discrete ID to a dense vector of dimensionality d. This matrix has

shape $|V| \times d$, where $|V|$ is the vocabulary size, i.e. the total number of distinct tokens that the model can represent [4]. When tokens correspond to words, the embedding can be interpreted as capturing semantic and syntactic properties of those words. Intuitively, words with similar meanings, such as "big" and "large," may be represented by vectors that are close to one another in the embedding space [4].

- **Positional Encoding**: Since the Transformer architecture processes sequences in parrallel, it does not inherently encode the order of tokens, so a positional encoding is added to the token embeddings to ensure that the position of each token in the sequence is represented. This is done by adding a positional encoding vector to each token embedding [4].

- **Input Formation**: For a sequence of N tokens, the individual d-dimensional vector embeddings are placed into a single matrix X of size $N \times d$. This matrix serves as the input for the model [4].

2. Transformer Blocks:

Each block consits of a multi-head self-attention mechanism followed by a feedforward neural network, with each sublayer followed by a layer normalisation and a residual connection [4].

A useful mental model is that each time the input passes through a sub-layer, the model updates the token representations so that they better capture the information relevant to the task at hand. For example, in an NLP context, the model may refine the representation of a word so that it better reflects its meaning in the context of the sentence [4].

- **Layer Normalisation**:

  For training stability, layer normalisation is applied to ensure the values in the sublayers remain within a reasonable range.

  For a vector x, it is calculated as:

$$\text{LayerNorm}(x) = \gamma \odot \frac{x - \mu}{\sigma} + \beta$$

  where μ is the mean, σ the standard deviation of the vector's elements, and γ and β are learned parameters that scale and shift the normalized output. [4].

- **Multi-Head Self-Attention**:

  This is the core mechanism that allows the model incorporate information from all tokens in the sequence when processing each token [4]. Under the interpretation that each sub-layer progressively refines word representations, the self-attention mechanism allows the representation of each word to be updated in light of the representations of all other words in the sequence. For example, if the word "fly" is preceded by the word "the", the model may adjust its representation in a way that reflects the nominal sense of "fly" (the insect) rather than the verbal sense. Each input vector x_i is mapped into three learned representations: a Query (Q), a Key (K), and a Value (V).

The attention for a single head is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

The scaling factor $\sqrt{d_k}$ prevents the dot products from growing too large and causing numerical instability. In causal (left-to-right) models, such as predicting the next word in a sentence, or the next position in a trajectory, a mask is applied to ensure that tokens can only attend to the past, preventing the model from "cheating" by looking at future tokens during training [4].

- **Feedforward Neural Network**:

  After the attention layer incorporates information across tokens, the Feedforward Network (FFN) processes each token position independently and in parallel [4]. It is usually a two-layer network with a non-linear activation such as ReLU:

  $$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

  This layer allows the model to perform more complex transformations of the contextualised information gathered by the attention heads [4].

  Returning to the intuitive picture introduced earlier, the attention mechanism can be viewed as first updating each word representation using information from the other words in the sequence. The FFN then further refines each of these updated representations independently at each position, using transformations learned from many training sequences.

3. Prediction Head:

After the data passes through the final transformer block, the final representations are passed to the next-token prediction head, which converts them into a probability distribution over possible next tokens in the sequence.

- **Output Projection**: Let h_t denote the final hidden representation produced after the model has processed the prefix up to position t. This d-dimensional vector summarises the information in the observed prefix and is then projected back into the vocabulary space, often by multiplying it by the transpose of the embedding matrix E^T. When the same embedding matrix is reused in this way, the technique is known as weight tying [4].

- **Softmax**: The resulting logits are passed through a softmax function to produce a probability distribution over the next token:

  $$\mathbf{p}_{t+1} = \text{softmax}(h_t E^T)$$

  where $\mathbf{p}_{t+1}$ is the probability distribution over possible values of the next token z_{t+1} given the prefix $z_{\leq t}$ [4].

In general, this means that the model assigns a probability to each possible next token given the preceding trajectory prefix. In an NLP setting, where tokens correspond to words, this can be interpreted as the model estimating which word is most likely to come next in the sentence. The next token is then selected according to a decoding strategy such as greedy decoding, sampling, or beam search.

Methodology

Problem Formulation

The objective of this work is to model maze planning as autoregressive inference in a learned conditional trajectory model trained through next-token prediction. Given a maze layout, a start state, a goal state, and a trajectory prefix, the model must predict the next token in the trajectory sequence.

Since a trajectory $\boldsymbol{\tau} = (z_1, z_2, \ldots, z_{2T})$ consists of alternating state and action tokens $(s_1, a_1, s_2, a_2, \ldots, s_T, a_T)$, and the state immediately following an action is fully determined by the action and the preceding state, supervision is applied only at action-token positions, so the learning problem is more precisely to predict the next action conditioned on the maze layout, the start state, the goal state, and the previously observed trajectory prefix.

Let m denote a maze layout identifier, let s_{start} and s_{goal} denote the start and goal states. The model therefore learns conditional predictions of the form

$$p(z_{t+1} \mid m, s_{\text{start}}, s_{\text{goal}}, z_{\leq t}),$$

or equivelantly since the model only needs to predict actions:

$$p(a_t \mid m, s_{\text{start}}, s_{\text{goal}}, s_1, a_1, \ldots, s_t),$$

with the goal of generating valid trajectories autoregressively at inference time.

A Transformer was selected as the generative model of choice because of its strong performance on sequence modelling tasks, the wide range of Python libraries available for working with it, and its computational efficiency when the architecture is kept small. The research question, however, is not whether a Transformer can represent such a distribution in principle, but how different decoding strategies affect the quality of the generated plans once the model has been trained. In particular, the methodology compares greedy decoding and beam search in terms of trajectory validity, goal-reaching success, path efficiency, and computational cost.

Dataset Construction

Since the goal is to train a Transformer to predict the next action given a trajectory prefix, the training data must consist of complete trajectories $\boldsymbol{\tau}$. A trajectory token sequence provides multiple supervised training contexts, since for each token position k, the prefix

$$(z_1, z_2, \ldots, z_{k-1})$$

can be used to predict the next token z_k. In this work, supervision is applied only at action-token positions, so the training data must also include the corresponding target action tokens against which the Transformer output is compared when computing the loss.

In addition to the trajectory tokens themselves, the model input must encode which maze the trajectory belongs to, as well as the associated start and goal positions. In the

implementation used here, each training example is therefore represented as a padded token sequence containing maze, start, goal, state, and action tokens. The dataset also includes a padding mask indicating which token positions are real and which correspond to padding. This is required because trajectories have different lengths and must be padded to a common sequence length for batch training in PyTorch.

For efficient training, these quantities are represented explicitly as matrices. The tokenised input sequences are stored in an input matrix X, the token-type information is stored in a token-type matrix T, the target outputs are stored in a label matrix Y, and the padding information is stored in a mask matrix M. The matrix T has the same shape as X, with each entry indicating whether the corresponding token is a maze token, start token, goal token, state token, action token, or padding token. In the label matrix, only action-token positions contain active targets, while all other positions are ignored during loss computation.

To realise the above representation, concrete design choices must be made regarding how states, actions, maze layouts, and the start and goal positions are encoded in matrix form. In this work, all of these are represented as discrete integer tokens. Distinct token ranges are reserved for different token classes by defining token offsets, so that maze tokens, state tokens, and action tokens occupy non-overlapping regions of the overall vocabulary.

States are encoded by assigning each grid location a unique integer index. This is done using the mapping

$$\text{state\_index} = \text{row} \times (\text{max columns}) + \text{column},$$

where *row* and *column* are the integer indices of the grid location, and *max columns* is the maximum number of columns across all maze layouts. This produces a unique integer identifier for each location in the global grid. A state token is then obtained by adding a fixed state-token offset to this index. Although this indexing scheme assigns identifiers to all grid locations in the global grid, including wall locations, only valid free-cell positions appear in the generated trajectory sequences.

Actions are also encoded as integer tokens. Since the trajectories are restricted to four-connected grid movement, the action vocabulary consists of the four moves up, down, left, and right, with each assigned its own unique token identifier.

In this work, eight different maze layouts are considered, corresponding to the layouts used in the D4RL PointMaze / Maze2D environment. The exact layout definitions can be found in the D4RL repository at https://github.com/Farama-Foundation/D4RL/blob/master/d4rl/pointmaze/maze_model.py.

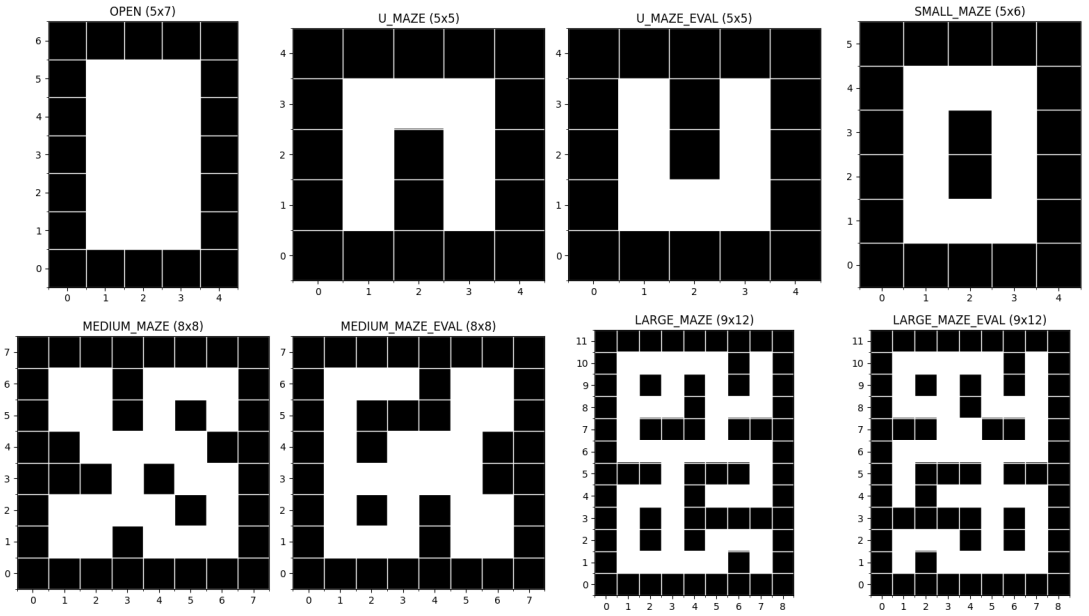

Figure 3: Maze layouts used from the D4RL PointMaze / Maze2D environment.

Each input sequence begins with a maze token, followed by a start-state token, a goal-state token, and the initial state token of the trajectory. The remainder of the sequence alternates between action tokens and subsequent state tokens. In this way, the start and goal positions are not stored separately as raw coordinate values, but are encoded using the same state-token mapping as other positions, while their functional role in the sequence is distinguished by their position and token type.

In the training data, the detailed wall structure of a maze is therefore not included explicitly in the token sequence. Instead, each trajectory is associated with a single integer maze token identifying which of the eight layouts it belongs to. The model must infer the relevant layout constraints from this maze identifier together with the observed trajectory tokens. This design choice gives a simple and efficient encoding scheme, but it also means that the model is tied to the set of maze layouts seen during training and cannot directly generalise to previously unseen layouts without additional representation of maze geometry.

As mentioned above, the input matrix X contains complete tokenised trajectories from which the Transformer learns. An important question is therefore how to obtain a sufficiently rich collection of complete trajectories. After discretising the maze layouts shown above, the number of free (non-wall) grid squares in a maze ranges from 7 to 47. If a maze has n free squares, then there are $n(n-1)$ ordered start-goal combinations, since the start and goal positions must be distinct.

For each such start-goal pair, an optimal trajectory is generated using breadth-first search (BFS). Here, optimal means that the resulting path minimises the number of adjacent grid moves required to connect the start and goal positions. Since movement is restricted to adjacent non-wall squares, BFS is sufficient to recover a shortest valid path in this discrete setting. Because the number of start-goal combinations for each maze is relatively small, generating shortest paths for all pairs is computationally inexpensive.

The BFS output provides a sequence of states, that is, a sequence of grid positions forming a shortest path from the start state to the goal state. The corresponding sequence of actions can then be computed directly from consecutive state differences. In this way, each start-goal pair yields a complete trajectory consisting of alternating state and action information, and these trajectories form the basis of the Transformer's training data.

The final form of each row $\boldsymbol{x}$ of the input matrix X is therefore as follows.

$\boldsymbol{x}_1$ is an integer token identifying the maze layout.

$\boldsymbol{x}_2$ is an integer token identifying the start state.

$\boldsymbol{x}_3$ is an integer token identifying the goal state.

$\boldsymbol{x}_4$ is an integer token identifying the initial trajectory state, which in this implementation is equal to the start state.

$\boldsymbol{x}_5$ is an integer token identifying the first action in the trajectory.

$\boldsymbol{x}_6$ is an integer token identifying the second state in the trajectory.

The remaining entries of $\boldsymbol{x}$ then alternate between action tokens and subsequent state tokens until the trajectory terminates at the goal. Any unused entries at the end of the row are filled with padding tokens so that all rows of X have the same length.

Transformer Training

A small causal Transformer was trained on the tokenised trajectory dataset described above. In the implementation used here, the model receives the input token matrix X, the token-type matrix T, and the padding mask M. Token embeddings and token-type embeddings are summed, positional encodings are added, and the resulting sequence is processed by a stack of causal self-attention layers so that each token can depend only on the current and preceding tokens in the sequence.

The model was kept small for faster training. The architecture used a hidden dimension of 64, 4 attention heads, 2 Transformer layers, a feedforward dimension of 128, and dropout of 0.1. Training was performed using the AdamW optimiser with learning rate 3×10^{-4} and weight decay 10^{-4}.

The training objective was cross-entropy loss applied to next-token prediction, with supervision active only at action-token positions. Cross-entropy was chosen because the target at each supervised position is a discrete action token drawn from a finite vocabulary, so the learning problem is naturally one of categorical prediction. Non-action positions in the label matrix Y were ignored during loss computation. The model was therefore trained to predict the next action conditioned on the maze token, the start token, the goal token, and the trajectory prefix observed so far.

References

[1] P. E. Hart, N. J. Nilsson, and B. Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*,

4(2):100–107, 1968.

[2] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

[3] Michael Janner, Qiyang Li, and Sergey Levine. Offline Reinforcement Learning as One Big Sequence Modeling Problem. *Advances in Neural Information Processing Systems*, 2021.

[4] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3rd edition. Online manuscript released January 6, 2026. https://web.stanford.edu/~jurafsky/slp3/ed3book_jan26.pdf

[5] Ian Goodfellow, Yoshua Bengio and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[6] Diaeddin Alarnaouti, George Baryannis, Mauro Vallati. *Reformulation techniques for automated planning: a systematic review. The Knowledge Engineering Review*, 38, 2023.